Reproducing and Improving TypiClust

Lucas Lobo - k23075501

1 Introduction

Deep learning requires large labelled datasets, yet annotation is costly. Active Learning (AL) mitigates this by iteratively selecting informative unlabelled examples, but traditional uncertainty-based methods rely on a trained model, creating a cold start problem. Hacohen et al. [1] identified that the optimal AL strategy depends on budget size: typical examples should be queried first, with uncertainty-based methods taking over once sufficient labels exist. They proposed TypiClust, which selects typical points from K-Means clusters in a self-supervised representation space. However, typicality alone can produce redundant selections. We reproduce TypiClust on CIFAR-10 across all three evaluation frameworks with 8 AL baselines, and propose a hybrid modification incorporating diversity-aware selection.

2 Methodology

2.1 TypiClust Pipeline

Our implementation follows the TPC_{RP} variant for initial pool selection ($L_0 = \emptyset$). Following the paper (Appendix F.1), we train a SimCLR [2] encoder (ResNet-18, 500 epochs) from scratch on CIFAR-10, using SGD (lr=0.4, momentum 0.9, cosine schedule, batch 512, weight decay 10^{-4}) and L2-normalised 512-d embeddings as the representation space. The resulting linear probe achieves 89.6% on the full training set. The algorithm proceeds as: (1) cluster embeddings with K-Means ($K = B$), (2) from each cluster, select the point with highest typicality:

$$\text{Typicality}(x) = \left(\frac{1}{k} \sum_{x_i \in \text{KNN}(x)} \|x - x_i\|_2 \right)^{-1} \tag{1}$$

where $k = 20$ neighbours. We evaluate across all three frameworks: (i) fully supervised (ResNet-18 trained from scratch), (ii) SS embedding (linear probe on frozen embeddings), and (iii) semi-supervised (pseudo-labelling), comparing against 8 baselines: Random, Uncertainty, Margin, Entropy, CoreSet [4], BADGE, BALD, and DBAL.

2.2 Proposed Modification

The paper's theoretical analysis (Section 2) shows that in the low-budget regime, selecting from high-density regions (R_1) is optimal, requiring both *max density* (typicality) and *diversity*. TypiClust enforces diversity via clustering, but purely typical selections can still be spatially redundant within clusters. To address this, we propose:

Hybrid Score Formula. We replace the pure typicality criterion with a hybrid score:

$$\text{score} = \alpha \cdot \hat{t} + (1 - \alpha) \cdot \hat{d} \tag{2}$$

where $\hat{t}$ is the normalised typicality and $\hat{d}$ is the normalised minimum distance to already-selected points, both scaled to $[0, 1]$ within each cluster. Selection proceeds sequentially: the most typical point overall is chosen first, and each subsequent point is scored considering distance from already-selected points.

Dimensionality Reduction. High-dimensional embeddings (512-d) can suffer from the curse of dimensionality [5], where distance metrics become less meaningful. We apply PCA and UMAP [3] to reduce dimensionality before clustering. PCA preserves global variance structure while UMAP captures nonlinear manifold geometry. Therefore, we sweep both to find the optimal configuration.

Alpha Tuning. α controls the typicality-diversity trade-off: $\alpha = 1$ recovers original TypiClust, $\alpha = 0$ selects purely for distance. We sweep α across $[0, 1]$.

3 Results

3.1 Paper Results and Reproduction

Table 1 compares the paper's results to our reproduction on CIFAR-10 ($B = 10$). In the SS embedding framework, our TypiClust achieves +9.8% over random, closely matching the paper's ∼+10%. Our slightly higher absolute values (42% vs ∼38% random) likely reflect minor differences in SimCLR training code vs the paper's Van Gansbeke et al. implementation. The paper additionally reports 93.2% in the semi-supervised setting (FlexMatch), which we approximate with pseudo-labelling (see Limitations).

	Random	TPC_{RP}	Δ	Source
Paper: Supervised	∼18%	∼28%	+10%	Fig. 4
Paper: SS Embed.	∼38%	∼48%	+10%	Fig. 5
Paper: Semi-sup.	53.8%	93.2%	+39.4%	Fig. 6
Ours: Supervised	15.4%	12.9%	−2.5%	fw1
Ours: SS Embed.	42.1%	51.8%	**+9.8%**	fw2
Ours: Semi-sup.	12.4%	15.8%	+3.4%	fw3

Table 1: Paper vs our reproduction ($B = 10$). SS embedding closely matches. The fully supervised gap at $B = 10$ is due to 9/10 class coverage (see Limitations), semi-supervised uses pseudo-labelling vs FlexMatch.

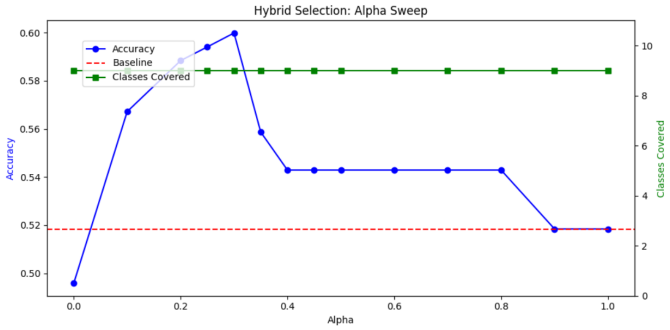

Figure 1: Hybrid selection on CIFAR-10 ($B = 10$): accuracy and class coverage vs. α. Dashed line shows baseline TypiClust.

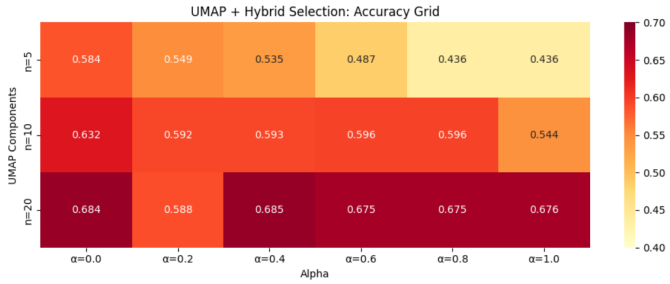

Figure 2: UMAP + hybrid accuracy grid on CIFAR-10 ($B = 10$) across UMAP dimensions and α.

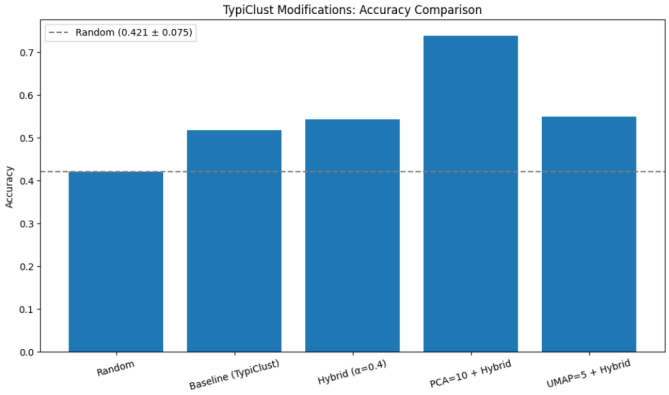

Figure 3: Accuracy comparison across all selection methods on CIFAR-10 ($B = 10$). Dashed line shows random baseline mean.

Method	Accuracy	Classes
Random (30 seeds)	$(42.07 \pm 7.45)\%$	–
TypiClust (baseline)	$(51.84 \pm 0.00)\%$	9
Hybrid ($\alpha = 0.4$)	$(54.29 \pm 0.00)\%$	9
UMAP-20 + Hybrid	$(68.51 \pm 3.62)\%$	–
PCA-10 + Hybrid	$(\mathbf{73.91 \pm 0.00})\%$	10

Table 2: Selection strategies on CIFAR-10 ($B = 10$).

3.2 Our Results

In the SS embedding framework, multi-round experiments across 5 AL rounds with all 8 baselines confirm that TypiClust dominates (51.9% at $B = 10$, rising to 81.6% at $B = 40$), while Margin overtakes at $B = 50$ (82.4% vs 80.8%), reproducing the phase transition (Appendix, Figure 6). In the fully supervised framework, all methods cluster between 13–19% at $B = 10$ with high variance, consistent with the paper's finding that training a network on 10 examples is inherently noisy. In semi-supervised, TypiClust's advantage is amplified at $B = 20$ (67.2% vs random 28.4%), confirming the paper's key result.

The alpha sweep (Figure 1) shows that hybrid selection ($\alpha = 0.4$) raises accuracy to 54.29%. The best result is PCA ($n = 10$, $\alpha = 0.55$): 73.91% with full class coverage (10/10), a +22.07% improvement over baseline (Figure 3). PCA outperforms UMAP because linear projection preserves the well-structured embedding geometry. UMAP's nonlinear distortion at low dimensions discards discriminative information, though it improves at $n = 20$ (68.5%, Figure 2).

We also evaluate over-clustering ($K > B$) with both PCA and UMAP. Over-clustering consistently degrades performance, validating the paper's $K = B$ design.

4 Statistical Analysis

The random baseline 95% CI is $[39.4\%, 44.7\%]$, TypiClust at 51.84% falls well above this, confirming a significant reproduction. For the modification, we compare each method (10 seeds) against the random baseline (30 seeds) using two-sample t-tests. All achieve significance: baseline ($t = 4.04$, $p < 0.001$, $d = 1.85$), hybrid ($t = 5.05$, $p < 0.001$, $d = 2.32$), PCA + hybrid ($t = 13.17$, $p < 0.001$, $d = 6.04$), UMAP + hybrid ($t = 9.78$, $p < 0.001$, $d = 4.19$). With Bonferroni correction ($\alpha_{\text{adj}} = 0.0125$), all remain significant.

5 Conclusion

We reproduce TypiClust on CIFAR-10 across three evaluation frameworks with 8 AL baselines. In the SS embedding framework, TypiClust outperforms all baselines at low budgets (+9.77% over random at $B = 10$) and dominates through $B = 40$, while a phase transition occurs where Margin surpasses it at $B = 50$. Our hybrid modification combining typicality with diversity-aware selection and PCA dimensionality reduction improves accuracy from 51.84% to 73.91% (+22.07%), achieving full class coverage.

Limitations. Our SimCLR merges two classes (dog/horse) into one cluster, limiting TypiClust to 9/10 coverage. Causing issues for fully supervised but manageable for SS embedding. Fully supervised uses 200 epochs/5 reps (vs paper's Munjal framework). Semi-supervised uses pseudo-labelling vs FlexMatch (infeasible: WideResNet-28, 400k iterations). **Code:** `https://github.com/LucasPRLobo/ml-cw2`

References

[1] G. Hacohen, A. Dekel, and D. Weinshall. Active learning on a budget: Opposite strategies suit high and low budgets. *ICML*, PMLR 162, 2022.

[2] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. *ICML*, pp. 1597–1607, 2020.

[3] L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

[4] O. Sener and S. Savarese. Active learning for convolutional neural networks: A core-set approach. *ICLR*, 2018.

[5] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. *ICDT*, pp. 420–434, 2001.

Additional Plots

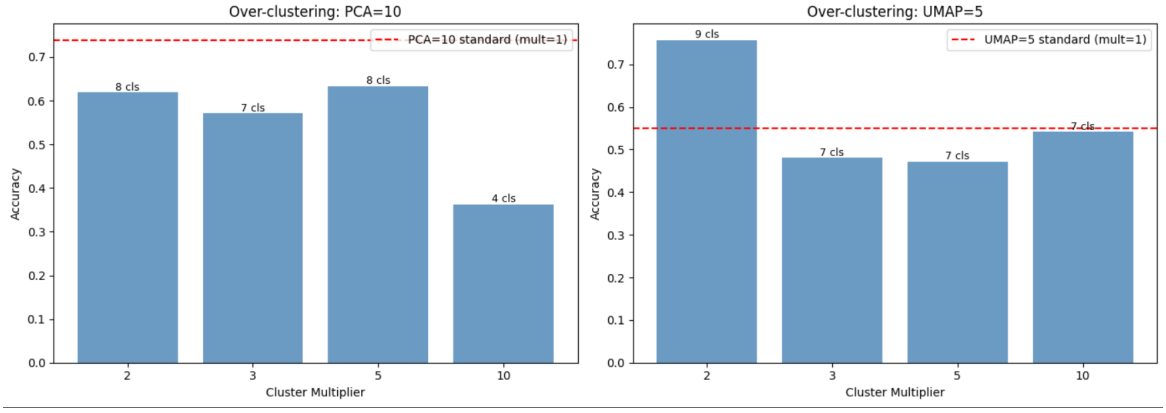

Figure 4: Over-clustering ablation for PCA-10 and UMAP-5. Dashed lines: standard $K = B$ result. Over-clustering consistently degrades performance.

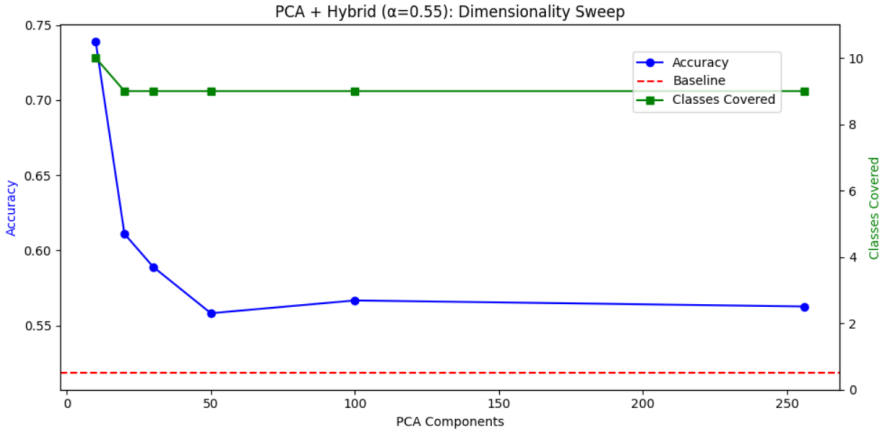

Figure 5: PCA + hybrid ($\alpha = 0.55$): accuracy and class coverage across PCA dimensions.

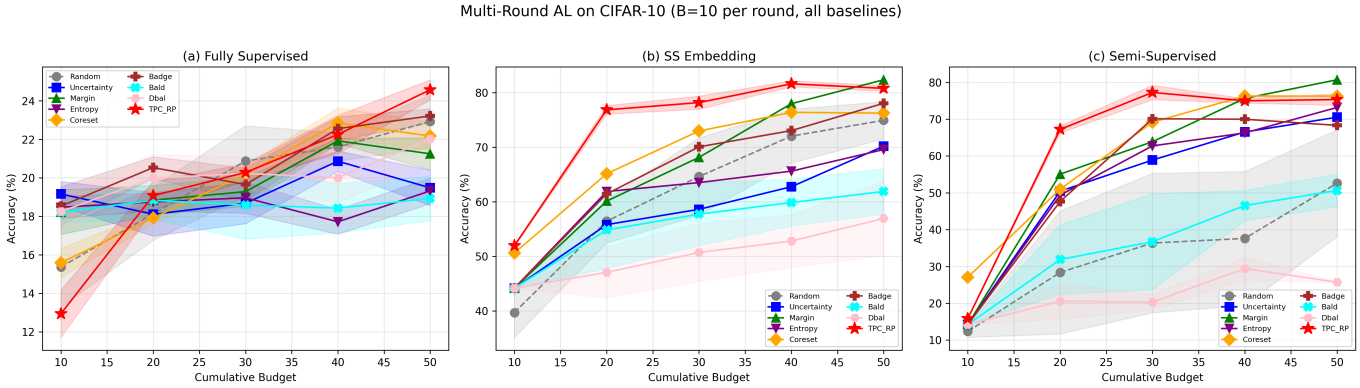

Figure 6: Multi-round AL comparison on CIFAR-10 across all three evaluation frameworks. (a) Fully supervised: all methods perform similarly (13–25%). TypiClust underperforms at $B = 10$ due to 9/10 class coverage but recovers at $B = 50$. (b) SS embedding: TypiClust dominates from $B = 10$ through $B = 40$. Margin overtakes at $B = 50$, reproducing the phase transition from [1]. (c) Semi-supervised (pseudo-labelling): TypiClust's advantage is amplified at $B = 20$ (67.2% vs random 28.4%). Margin overtakes at $B = 50$ (80.7%).

original_algorithm

March 24, 2026

1 Original TypiClust Algorithm

Reproduction of TPC_RP (SimCLR + K-Means + Typicality Selection) on CIFAR-10 with B=10.

Note: The SimCLR encoder (ResNet-18, 200 epochs) is trained from scratch in a separate notebook (`SimCLR_implementation.ipynb`). The training is split out so the algorithm can be run without retraining each time. The saved weights are loaded below.

1.1 1. Setup & Data Loading

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

[2]:
```
# Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
```

1.2 2. SimCLR Encoder (Representation Learning)

[3]:
```
class ResNet18Encoder(nn.Module):
    """SimCLR encoder with ResNet-18 backbone adapted for CIFAR-10 (32x32)."""
    def __init__(self):
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
 ↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            h = torch.nn.functional.normalize(h, dim=1)  # L2 normalize (paper␣
 ↪F.1)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

[4]:
```
# Load pretrained SimCLR encoder and extract embeddings
encoder = load_encoder('./simclr_encoder_500ep.pth')
embeddings, labels = extract_embeddings(encoder, trainloader)
test_embeddings, test_labels = extract_embeddings(encoder, testloader)
print(f"Train embeddings: {embeddings.shape}")
print(f"Test embeddings: {test_embeddings.shape}")
```

```
Train embeddings: (50000, 512)
Test embeddings: (10000, 512)
```

1.3 3. Typicality Computation

[5]:
```
def compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1


typicality = compute_typicality(embeddings, k=20)
print(f"Typicality shape: {typicality.shape}")
print(f"Typicality range: [{typicality.min():.4f}, {typicality.max():.4f}]")
```

```
Typicality shape: (50000,)
Typicality range: [1.0799, 18.4421]
```

1.4 4. Clustering (K-Means, K=B)

[6]:
```
def cluster_standard(embeddings, budget, random_state=42):
    """K-Means clustering with K = budget."""
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_


budget = 10
cluster_labels = cluster_standard(embeddings, budget)
print(f"Cluster sizes: {np.bincount(cluster_labels)}")
```

```
Cluster sizes: [5314 3763 4912 5582 4727 2818 4972 4982 6392 6538]
```

1.5 5. Selection (Max Typicality)

[7]:
```
def select_max_typicality(typicality, cluster_labels, budget):
    """Select the most typical point from each cluster."""
    selected = []
    for cluster in range(budget):
        cluster_idx = np.where(cluster_labels == cluster)[0]
        best = cluster_idx[np.argmax(typicality[cluster_idx])]
        selected.append(best)
    return np.array(selected)


selected_indices = select_max_typicality(typicality, cluster_labels, budget)
print(f"Selected indices: {selected_indices}")
print(f"Selected labels: {labels[selected_indices]}")
```

```
print(f"Classes covered: {len(np.unique(labels[selected_indices]))}")
```

```
Selected indices: [38224 29916 18943 38604 42923 39686 30062  7617 48591 37173]
Selected labels: [0 7 9 7 6 2 1 8 4 3]
Classes covered: 9
```

1.6 6. Evaluation (Linear Probe)

[8]:
```
def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
↪test_labels):
    """Train a linear probe on selected examples and evaluate on test set."""
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(sel_emb, sel_lab)
    preds = clf.predict(test_embeddings)
    accuracy = accuracy_score(test_labels, preds)
    n_classes = len(np.unique(sel_lab))
    return {'accuracy': accuracy, 'n_classes_covered': n_classes}


results = evaluate_selection(selected_indices, embeddings, labels,␣
↪test_embeddings, test_labels)
print(f"\n=== Original TypiClust Results ===")
print(f"Accuracy: {results['accuracy']:.4f}")
print(f"Classes covered: {results['n_classes_covered']}/10")
```

```
=== Original TypiClust Results ===
Accuracy: 0.5184
Classes covered: 9/10
```

1.7 7. Random Baseline Comparison

[9]:
```python
# Random baseline (30 seeds)
random_accs = []
for seed in range(30):
    rng = np.random.RandomState(seed)
    idx = rng.choice(len(embeddings), size=budget, replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(embeddings[idx], labels[idx])
    preds = clf.predict(test_embeddings)
    random_accs.append(accuracy_score(test_labels, preds))

random_accs = np.array(random_accs)
print(f"Random Baseline: {random_accs.mean():.4f} +/- {random_accs.std():.4f}")
print(f"TypiClust: {results['accuracy']:.4f}")
print(f"Improvement: +{results['accuracy'] - random_accs.mean():.4f}")
```

```
Random Baseline: 0.4207 +/- 0.0745
TypiClust: 0.5184
Improvement: +0.0977
```

modified_algorithm

March 24, 2026

1 Modified TypiClust Algorithm

Best configuration: PCA (n=10) + Hybrid Selection (alpha=0.55) on CIFAR-10 with B=10.

1.1 1. Setup & Data Loading

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
```

[2]:
```
# Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
```

1.2 2. SimCLR Encoder (Representation Learning)

[3]:
```
class ResNet18Encoder(nn.Module):
    """SimCLR encoder with ResNet-18 backbone adapted for CIFAR-10 (32x32)."""
    def __init__(self):
```

```
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            h = torch.nn.functional.normalize(h, dim=1)  # L2 normalize (paper␣
↪F.1)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

[4]:
```
# Load pretrained SimCLR encoder and extract embeddings
encoder = load_encoder('./simclr_encoder_500ep.pth')
embeddings, labels = extract_embeddings(encoder, trainloader)
test_embeddings, test_labels = extract_embeddings(encoder, testloader)
print(f"Train embeddings: {embeddings.shape}")
print(f"Test embeddings: {test_embeddings.shape}")
```

```
Train embeddings: (50000, 512)
Test embeddings: (10000, 512)
```

1.3 3. PCA Dimensionality Reduction (Modification 1)

[5]:
```
def preprocess_pca(embeddings, n_components=10):
    """Reduce embedding dimensionality with PCA before clustering."""
    pca = PCA(n_components=n_components)
    transformed = pca.fit_transform(embeddings)
    return transformed, pca


# Apply PCA (n=10) to train and test embeddings
proc_embeddings, pca_model = preprocess_pca(embeddings, n_components=10)
proc_test = pca_model.transform(test_embeddings)
print(f"PCA train embeddings: {proc_embeddings.shape}")
print(f"PCA test embeddings: {proc_test.shape}")
print(f"Explained variance ratio: {pca_model.explained_variance_ratio_.sum():.
 ↪4f}")
```

```
PCA train embeddings: (50000, 10)
PCA test embeddings: (10000, 10)
Explained variance ratio: 0.4153
```

1.4 4. Typicality Computation

[6]:
```
def compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1


typicality = compute_typicality(proc_embeddings, k=20)
print(f"Typicality shape: {typicality.shape}")
print(f"Typicality range: [{typicality.min():.4f}, {typicality.max():.4f}]")
```

```
Typicality shape: (50000,)
Typicality range: [2.6835, 86.8538]
```

1.5 5. Clustering (K-Means, K=B)

[7]:
```
def cluster_standard(embeddings, budget, random_state=42):
    """K-Means clustering with K = budget."""
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_
```

```
budget = 10
cluster_labels = cluster_standard(proc_embeddings, budget)
print(f"Cluster sizes: {np.bincount(cluster_labels)}")
```

```
Cluster sizes: [3803 4182 6327 4949 4946 4638 6124 5681 6018 3332]
```

1.6 6. Hybrid Selection (Modification 2)

[8]:
```
def select_hybrid(embeddings, typicality, cluster_labels, budget, alpha=0.55):
    """
    Hybrid selection: score = alpha * typicality + (1-alpha) * min_distance.
    Selects sequentially, balancing representativeness with spatial diversity.
    """
    selected = []

    # Order clusters by max typicality (most typical cluster first)
    cluster_order = []
    for c in range(budget):
        cluster_idx = np.where(cluster_labels == c)[0]
        cluster_order.append((c, np.max(typicality[cluster_idx])))
    cluster_order.sort(key=lambda x: x[1], reverse=True)

    # First point: most typical overall
    first_cluster = cluster_order[0][0]
    cluster_idx = np.where(cluster_labels == first_cluster)[0]
    best = cluster_idx[np.argmax(typicality[cluster_idx])]
    selected.append(best)

    # Subsequent points: hybrid score within each cluster
    for cluster in cluster_order[1:]:
        cluster_idx = np.where(cluster_labels == cluster[0])[0]
        candidate_embeddings = embeddings[cluster_idx]

        # Compute min distance to already-selected points
        dist = np.linalg.norm(
            candidate_embeddings[:, None] - embeddings[selected], axis=2
        )
        min_distance = dist.min(axis=1)

        # Normalise both terms to [0, 1] within this cluster
        norm_distance = (min_distance - min_distance.min()) / (
            min_distance.max() - min_distance.min() + 1e-8
        )
        norm_typicality = (typicality[cluster_idx] - typicality[cluster_idx].
↪min()) / (
            typicality[cluster_idx].max() - typicality[cluster_idx].min() + 1e-8
        )
```

```
        # Hybrid score
        score = alpha * norm_typicality + (1 - alpha) * norm_distance
        best = cluster_idx[np.argmax(score)]
        selected.append(best)

    return np.array(selected)


selected_indices = select_hybrid(proc_embeddings, typicality, cluster_labels,␣
 ↪budget, alpha=0.55)
print(f"Selected indices: {selected_indices}")
print(f"Selected labels: {labels[selected_indices]}")
print(f"Classes covered: {len(np.unique(labels[selected_indices]))}")
```

```
Selected indices: [33063  3766 23733 10212 28528 36746 29705 30391 34326 15152]
Selected labels: [1 9 5 0 7 8 3 2 6 4]
Classes covered: 10
```

1.7 7. Evaluation (Linear Probe)

[9]:
```
def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
 ↪test_labels):
    """Train a linear probe on selected examples and evaluate on test set."""
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(sel_emb, sel_lab)
    preds = clf.predict(test_embeddings)
    accuracy = accuracy_score(test_labels, preds)
    n_classes = len(np.unique(sel_lab))
    return {'accuracy': accuracy, 'n_classes_covered': n_classes}


results = evaluate_selection(selected_indices, proc_embeddings, labels,␣
 ↪proc_test, test_labels)
print(f"\n=== Modified TypiClust Results (PCA-10 + Hybrid alpha=0.55) ===")
print(f"Accuracy: {results['accuracy']:.4f}")
print(f"Classes covered: {results['n_classes_covered']}/10")
```

```
=== Modified TypiClust Results (PCA-10 + Hybrid alpha=0.55) ===
Accuracy: 0.7391
Classes covered: 10/10
```

1.8 8. Comparison with Baseline

[10]:
```
# Random baseline (30 seeds) on PCA-reduced embeddings
random_accs = []
for seed in range(30):
    rng = np.random.RandomState(seed)
    idx = rng.choice(len(proc_embeddings), size=budget, replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(proc_embeddings[idx], labels[idx])
    preds = clf.predict(proc_test)
    random_accs.append(accuracy_score(test_labels, preds))

random_accs = np.array(random_accs)

# Original TypiClust baseline (no PCA, no hybrid) on raw embeddings
from sklearn.cluster import KMeans as KM
km = KM(n_clusters=budget, random_state=42, n_init=10)
km.fit(embeddings)
baseline_typ = compute_typicality(embeddings, k=20)
baseline_sel = []
for c in range(budget):
    cidx = np.where(km.labels_ == c)[0]
    baseline_sel.append(cidx[np.argmax(baseline_typ[cidx])])
baseline_sel = np.array(baseline_sel)
clf = LogisticRegression(max_iter=1000, C=100)
clf.fit(embeddings[baseline_sel], labels[baseline_sel])
baseline_acc = accuracy_score(test_labels, clf.predict(test_embeddings))

print(f"Random Baseline: {random_accs.mean():.4f} +/- {random_accs.std():.4f}")
print(f"Original TypiClust: {baseline_acc:.4f}")
print(f"Modified (PCA-10 + Hybrid): {results['accuracy']:.4f}")
print(f"Improvement over baseline: +{results['accuracy'] - baseline_acc:.4f}")
```

```
Random Baseline: 0.4832 +/- 0.0731
Original TypiClust: 0.5184
Modified (PCA-10 + Hybrid): 0.7391
Improvement over baseline: +0.2207
```

SimCLR_implementation

March 24, 2026

1 SimCLR Implementation

Training hyperparameters follow the paper (Appendix F.1, Van Gansbeke et al. 2020): - **Architecture:** ResNet-18 encoder + MLP projection head (512 → 128) - **Optimizer:** SGD, 0.9 momentum, weight decay 0.0001 - **Learning rate:** 0.4, cosine scheduler - **Batch size:** 512 - **Epochs:** 500 - **Augmentations:** Random resized crop, horizontal flip, color jitter, random grayscale - **Loss:** NT-Xent (=0.5) - **Embedding:** L2-normalized 512-d penultimate layer

[11]:
```
import torch
import torchvision.transforms as transforms
import torch.nn as nn
import torchvision
import torchvision.models as models

from torch.utils.data import DataLoader
from tqdm import tqdm
```

1.0.1 Augmentation Transform

[12]:
```
class SimCLRTransform:
    def __init__(self):
        self.transform = transforms.Compose([
            transforms.RandomResizedCrop(size=32),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([transforms.ColorJitter(0.4, 0.4, 0.4, 0.
 ↪1)], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
        ])

    def __call__(self, x):
        return self.transform(x), self.transform(x)
```

1.0.2 ResNet-18 Encoder

[13]:
```
class ResNet18Encoder(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18Encoder, self).__init__()
        self.resnet = models.resnet18(pretrained=False)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()  # Remove maxpool layer for CIFAR-10
        self.resnet.fc = nn.Identity()  # Remove fully connected layer for␣
↪feature extraction

    def forward(self, x):
        return self.resnet(x)
```

1.0.3 Projection Head

[14]:
```
class ProjectionHead(nn.Module):
    def __init__(self, input_dim=512, hidden_dim=128, output_dim=128):
        super(ProjectionHead, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

1.0.4 NT_Xent Loss

[15]:
```
def nt_xent_loss(z_i, z_j, temperature=0.5):
    batch_size = z_i.size(0)
    z = torch.cat([z_i, z_j], dim=0)  # Concatenate positive pairs

    z = nn.functional.normalize(z, dim=1)  # L2 Norm
    similarity_matrix = torch.matmul(z, z.T)/temperature  # Compute similarity␣
↪matrix

    mask = torch.eye(2 * batch_size, dtype=torch.bool).to(z.device)
    similarity_matrix = similarity_matrix[~mask].view(2 * batch_size, -1)  #␣
↪Remove self-similarity

    targets = torch.cat([
        torch.arange(batch_size -1, 2*batch_size -1),
        torch.arange(0, batch_size)
```

```
    ]).to(z.device)

    loss = nn.CrossEntropyLoss()(similarity_matrix, targets)
    return loss
```

1.0.5 Import Dataset

[ ]:
```
simclr_trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                               download=True,␣
 ↪transform=SimCLRTransform())
simclr_loader = DataLoader(simclr_trainset, batch_size=512, shuffle=True,␣
 ↪num_workers=2, drop_last=True)
```

1.0.6 Training Loop Setup

[ ]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# Initialize models
encoder = ResNet18Encoder().to(device)
projection_head = ProjectionHead().to(device)

# Optimizer - matches paper: SGD, 0.9 momentum, lr=0.4, cosine schedule, weight␣
 ↪decay 0.0001
num_epochs = 500
optimizer = torch.optim.SGD(
    list(encoder.parameters()) + list(projection_head.parameters()),
    lr=0.4, momentum=0.9, weight_decay=0.0001
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,␣
 ↪T_max=num_epochs)

print(f'Training for {num_epochs} epochs')
print(f'Optimizer: SGD (lr=0.4, momentum=0.9, weight_decay=0.0001)')
print(f'Scheduler: CosineAnnealingLR')
```

1.0.7 Training Loop

[ ]:
```
for epoch in range(num_epochs):
    running_loss = 0.0
    loop = tqdm(simclr_loader, desc=f'Epoch [{epoch+1}/{num_epochs}]',␣
 ↪leave=False)
    for (view1, view2), _ in loop:
        view1, view2 = view1.to(device), view2.to(device)

        h_i = encoder(view1)
        h_j = encoder(view2)
```

```
        z_i = projection_head(h_i)
        z_j = projection_head(h_j)

        loss = nt_xent_loss(z_i, z_j)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        loop.set_postfix(loss=loss.item())

    scheduler.step()
    avg_loss = running_loss / len(simclr_loader)
    if (epoch + 1) % 50 == 0 or epoch == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Avg Loss: {avg_loss:.4f}, LR:␣
↪{scheduler.get_last_lr()[0]:.6f}')

# Save encoder weights
torch.save(encoder.state_dict(), 'simclr_encoder_500ep.pth')
print('Training complete. Encoder saved to simclr_encoder_500ep.pth')
```